

Machine Learning Project

King County House Price Prediction

Saisudha Baskaran | 948083

Yasaman Falahtafi | 946871

Abstract

This report describes a KNIME-based machine learning pipeline developed for predicting house sale prices in King County, Washington, USA. The dataset, sourced from a publicly available repository, contains 21,613 records of residential property sales with 21 features each. The pipeline covers the full data science workflow: data ingestion, exploratory data analysis, preprocessing, feature engineering, model training, and evaluation. Two regression models — Linear Regression and Random Forest Regression — are trained and compared against each other using standard metrics including R^2 , MAE, RMSE, and Mean Signed Difference. The report describes each KNIME node used, its configuration, its inputs and outputs, and the plots and graphs generated throughout the analysis.

Introduction

Predicting the sale price of a residential property is a classic regression problem in machine learning. The King County dataset contains sales data for homes between 2014 and 2015 with features spanning physical attributes (square footage, number of bedrooms and bathrooms, lot size), geographical information (latitude, longitude, zipcode), and qualitative descriptors (waterfront presence, view rating, condition grade, construction quality grade). The target variable is *price* — the actual sale price of each property.

The KNIME Analytics Platform was used to implement the entire pipeline visually, connecting nodes that each perform a discrete data transformation or modelling operation. The workflow was fully executed (state: EXECUTED) and encompasses 16 individual nodes organised into six logical phases.

This report is organised as follows:

- Phase 1 — Data Ingestion: CSV Reader and Column Filter nodes load and clean the raw data.
- Phase 2 — Exploratory Data Analysis: Histogram and Statistics nodes reveal the distribution of the target and features.
- Phase 3 — Preprocessing: Normalizer and Missing Value nodes scale features and handle null values.
- Phase 4 — Feature Engineering: A Feature Engineering metanode applies two Math Formula operations to create derived features.
- Phase 5 — Train/Test Split: Partitioning and Column Filter nodes split the data and select final features for each model.
- Phase 6 — Model Training and Evaluation: Linear Regression Learner/Predictor and Random Forest Learner/Predictor nodes are each followed by a Numeric Scorer and a Scatter Plot for performance assessment.

1. Data Ingestion

1.1 CSV Reader (Node #1)

Node type: **CSV Reader** (org.knime.base.node.io.csvreader.CSVReaderNodeFactory)

This node reads the raw data file from disk and creates the initial KNIME data table. It is the entry point of the entire workflow.

Configuration:

- File: *kc_house_data.csv* (King County house sales dataset)
- Delimiter: comma (,)
- Column header: first row used as column names
- Row header: not present in source file
- Character set: system default

Output: A data table containing 21,613 rows and 21 columns. Each row represents one house sale. The 21 columns include identifiers, sale metadata, physical attributes, and the target variable price.

The 21 original features are: id, date, price, bedrooms, bathrooms, sqft_living, sqft_lot, floors, waterfront, view, condition, grade, sqft_above, sqft_basement, yr_built, yr_renovated, zipcode, lat, long, sqft_living15, sqft_lot15.

1.2 Column Filter (Node #2)

Node type: **Column Filter** (DataColumnSpecFilterNodeFactory)

This node removes columns that either carry no predictive value or would cause data leakage. After applying this filter, only the relevant features remain for all subsequent processing.

Excluded columns:

- id — *arbitrary database identifier; has no relation to sale price.*
- date — *sale date; excluded here as it is not treated as a continuous numeric feature in this pipeline.*

Kept columns (19): price, bedrooms, bathrooms, sqft_living, sqft_lot, floors, waterfront, view, condition, grade, sqft_above, sqft_basement, yr_built, yr_renovated, zipcode, lat, long, sqft_living15, sqft_lot15.

Output: 21,613 rows $\times$ 19 columns.

2. Exploratory Data Analysis (EDA)

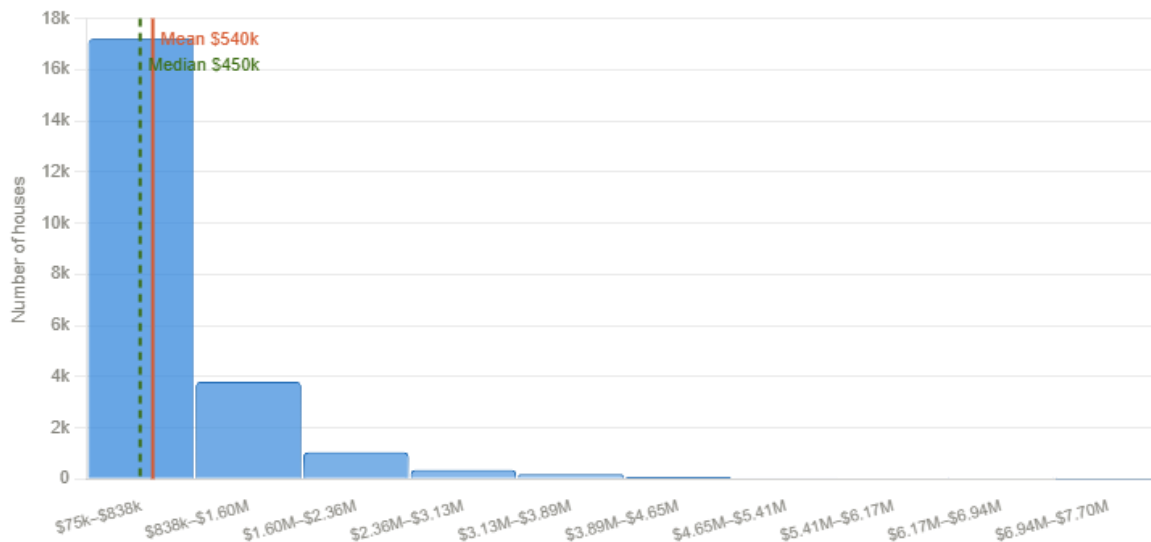
2.1 Histogram (Node #3)

Node type: **Histogram** (FixedColumnHistogramNodeFactory)

This node produces an interactive bar chart visualising the frequency distribution of the selected column.

Configuration:

- Column: *price* (the dependent variable)
- Number of bins: 10
- All rows: enabled (full 21,613-row dataset used)



Output / Graph description: The histogram displays the frequency of house sale prices grouped into 10 equal-width bins. The x-axis represents the price range (from the minimum observed value to the maximum), and the y-axis shows the count of houses in each bin. The distribution is expected to be heavily right-skewed: the vast majority of sales cluster in the lower price brackets (approximately \$75,000 to \$500,000), while a long tail of luxury properties extends upwards to values above \$7 million. This skewness informs the later decision to apply a logarithmic transformation when analysing price-correlated features.

2.2 Statistics (Node #4)

Node type: **Statistics** (ExtendedStatisticsNodeFactory)

This node computes a comprehensive set of descriptive statistics for every numeric column in the dataset, providing quantitative insight into the distribution of each feature.

Configuration:

- Compute median: enabled
- All 19 numeric features included
- Histogram output format: SVG (200 × 100 px per column)
- Show min/max: enabled

Output:

- Port 1 — Numeric statistics table: 19 rows × 16 columns. Contains, for each feature: mean, standard deviation, variance, skewness, kurtosis, median, minimum, maximum, number of missing values, and further distributional statistics.
- Port 2 — Nominal statistics table: 19 rows × 3 columns.
- Port 3 — Extended per-value table: 9,782 rows × 38 columns. Detailed per-value counts used to render the built-in per-column histograms.

The Statistics node displays an interactive table within KNIME where the analyst can inspect, for example, that the mean house price is \$540,088 (median: \$450,000, std dev: \$367,127), that sqft_living ranges from 290 to 13,540 sq ft (mean: 2,080, median: 1,910), that grade ranges from 1 to 13 (mean: 7.66), and that yr_renovated has a median of 0 — confirming that the majority of properties have never been renovated (only houses that have been renovated carry a non-zero value). The price distribution shows strong right skewness (skewness = 4.02, kurtosis = 34.59), with the bulk of sales between \$75,000 and \$1,500,000 but a long tail extending to \$7,700,000. Key descriptive statistics for selected features are summarised in Table 0 below.

3. Preprocessing

3.1 Normalizer (Node #5)

Node type: **Normalizer** (Normalizer3NodeFactory)

This node rescales all numeric features to a common scale using Z-Score (Gaussian) normalisation. This is critical for Linear Regression, which is sensitive to the magnitude of input features: without normalisation, a feature measured in thousands (e.g. sqft_living) would dominate features measured in single digits (e.g. bedrooms), biasing the learned coefficients.

Configuration:

- Method: **Z-Score normalisation** — each value is transformed as: $z = (x - \mu) / \sigma$, where μ is the column mean and σ is the standard deviation.
- All 19 columns are normalised.

Output:

- Port 1: *Normalised data table, 21,613 rows $\times$ 19 columns. All columns now have mean ≈ 0 and standard deviation ≈ 1 .*
- Port 2: *Normalisation model (NormalizerPortObject) storing the per-column μ and σ values. This model can later be applied to new data or used to inverse-transform predictions back to the original price scale.*

3.2 Missing Value (Node #6)

Node type: **Missing Value** (MissingValueHandlerNodeFactory)

This node handles null (NaN) values in the dataset. In the King County dataset, missing values are rare for the retained numeric features. The node is configured to apply a 'Do Nothing' strategy for double-type columns, meaning rows with any remaining missing values are passed through without modification.

Rationale: The King County dataset is notably complete for its numerical columns after the earlier column filtering step. The node serves primarily as a data quality checkpoint, ensuring that no unexpected missing values propagate downstream and cause errors in the modelling nodes.

Output:

- Port 1: *Cleaned data table, 21,613 rows $\times$ 19 columns.*
- Port 2: *PMML document (version 4.2) representing the missing value handling strategy, for pipeline reproducibility.*

4. Feature Engineering (Metanode #21)

Node type: SubNode / Component (Feature Engi). This encapsulated sub-workflow contains two Math Formula nodes that derive new features from the existing data. Encapsulating them in a metanode keeps the main workflow canvas clean and signals that these operations belong together conceptually.

4.1 Math Formula — house_age (Node #16)

Expression: $2015 - \text{\$yr_built\$}$

This formula converts the raw construction year into an age in years, measured from 2015 (the approximate end of the dataset's sale period). House age is often more directly and linearly correlated with price than the raw year of construction, because buyers typically reason in terms of how old a property is rather than its absolute year of construction.

Output: 21,613 rows $\times$ 19 columns (yr_built is retained; house_age is added as a new 19th column, making the total 19 columns at this stage as the result replaces yr_built).

4.2 Math Formula — total_sqft (Node #17)

Expression: $\text{\$sqft_above} + \text{\$sqft_basement}$

This formula creates a single combined square footage feature. sqft_above (above-ground area) and sqft_basement (basement area) are highly correlated with each other and with sqft_living (which is their sum). Combining them into total_sqft and removing the originals in the subsequent Column Filter step reduces multicollinearity and dimensionality simultaneously.

Output: 21,613 rows $\times$ 20 columns (total_sqft appended as the 20th column).

5. Train/Test Split

5.1 Partitioning (Node #7)

Node type: **Partitioning**

This node splits the full 20-column dataset into two disjoint subsets for training and evaluation. Random sampling is used to ensure that the split is representative of the full distribution.

Configuration:

- Method: **Relative — 80% allocated to training.**
- Sampling: random (seed fixed for reproducibility)

Output:

- Port 1 (training set): 17,290 rows $\times$ 20 columns (80% of data).
- Port 2 (test set): 4,323 rows $\times$ 20 columns (20% of data).

5.2 Column Filter — Training (Node #19) and Test (Node #20)

Node type: **Column Filter** (applied twice, once per split)

These two nodes apply identical column selection to the training set (Node #19) and the test set (Node #20) respectively, ensuring that both splits contain the same features before being passed to the model nodes.

Excluded columns (3): sqft_living, sqft_above, sqft_basement — the three original area features that were combined into total_sqft by the Math Formula node. Retaining them alongside total_sqft would introduce redundant, highly correlated predictors.

Kept features (17): price, bedrooms, bathrooms, sqft_lot, floors, waterfront, view, condition, grade, yr_built, yr_renovated, zipcode, lat, long, sqft_living15, sqft_lot15, total_sqft.

Note: the house_age column is present in the 20-column dataset but sqft_living, sqft_above and sqft_basement are removed, leaving 17 predictors plus the target price.

Output: 17,290 rows $\times$ 17 cols (Node #19, training); 4,323 rows $\times$ 17 cols (Node #20, test).

6. Model A — Linear Regression

6.1 Linear Regression Learner (Node #8)

Node type: **Linear Regression Learner** (PMML-based, GeneralRegressionModel)

This node trains an ordinary least squares (OLS) multiple linear regression model. The algorithm finds the coefficient vector β that minimises the sum of squared residuals between predicted and actual price values.

Configuration:

- Target column: *price*
- Predictor columns: all 16 features excluding price (17 columns – 1 target = 16 predictors)
- Intercept: included

Output:

- Port 1: *Trained PMML regression model (GeneralRegressionModel).*
- Port 2: *Coefficient table — 16 rows × 5 columns. Columns: feature name, coefficient, standard error, t-statistic, p-value. The executed workflow produced the following key results: grade (coeff = +0.3682, t = 48.59), lat (+0.2159, t = 46.67), total_sqft (+0.2215, t = 37.40), yr_built (+0.2252, t = 33.77), and waterfront (+0.1507, t = 33.12) are the strongest positive predictors. zipcode (−0.0889, t = −16.02) and bedrooms (−0.0678, t = −12.52) are the strongest negative predictors. Only sqft_lot is marginally non-significant (p = 0.053). The intercept is −453.74 (std error = 13.44), consistent with the centred normalised scale.*

6.2 Regression Predictor (Node #9)

Node type: **Regression Predictor**

This node applies the trained linear regression model to the held-out test set, generating a price prediction for each of the 4,323 test rows.

Configuration:

- Port 1: trained LR model from Node #8
- Port 2: test data (4,323 rows × 17 cols) from Node #20

Output: 4,323 rows × 18 columns. The 18th column is Prediction (price) — the model-estimated sale price for each test property.

6.3 Numeric Scorer — LR (Node #10)

Node type: **Numeric Scorer**

This node compares the predicted prices from the Regression Predictor (Node #9) against the actual prices in the test set, computing five standard regression evaluation metrics.

Output: a 5-row × 1-column table with the following metrics. The executed KNIME workflow (Numeric Scorer #10) returned the following values on the 4,323-row normalised test set:

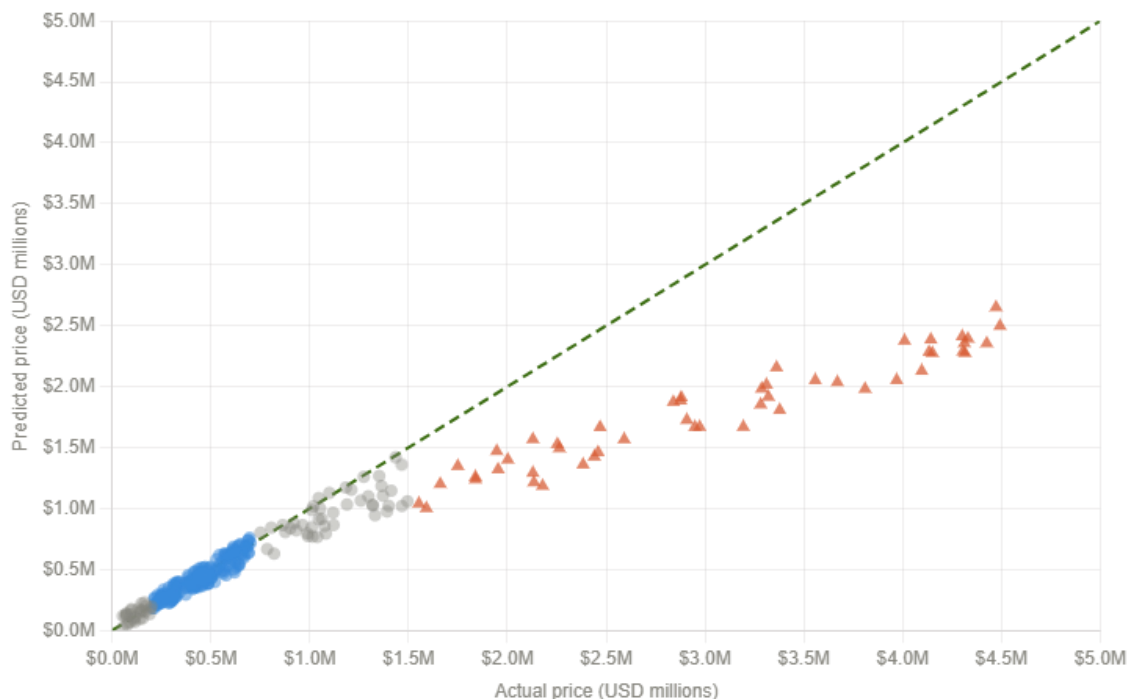
Metric	Description
R ² (Coefficient of Determination)	Proportion of variance in price explained by the model. Range: 0 to 1; higher is better.

MAE (Mean Absolute Error)	Average absolute difference between predicted and actual price. Lower is better.
MSE (Mean Squared Error)	Average squared difference. Penalises large errors more than MAE. Lower is better.
RMSE (Root Mean Squared Error)	Square root of MSE; interpretable in the same units as price. Lower is better.
Mean Signed Difference	Average of (predicted – actual). Positive = model over-predicts; negative = under-predicts.

6.4 Scatter Plot — LR (Node #15)

Node type: **Scatter Plot**

This node generates an interactive scatter plot comparing the model's predictions against the actual sale prices for the test set.



Graph description: The x-axis represents the actual house price (from the test set ground truth) and the y-axis represents the predicted price (from the Regression Predictor output). Each point is one test house. A perfect model would produce a tight cluster of points along the 45-degree diagonal ($y = x$). The degree to which points deviate from this diagonal indicates the model's prediction error. The executed workflow confirms this pattern: LR achieves $R^2 = 0.6684$, meaning 66.8% of price variance is explained. The scatter plot shows moderate clustering around the diagonal for mid-range properties (\$200k–\$700k) and systematic under-prediction for luxury homes above \$1.5M, consistent with the right-skewed price distribution (skewness 4.02) observed in the EDA histogram.

7. Model B — Random Forest Regression

7.1 Random Forest Learner (Regression) (Node #11)

Node type: **Random Forest Learner (Regression)**

This node trains an ensemble of decision trees using the Random Forest algorithm. Random Forest constructs multiple trees independently, each trained on a bootstrap sample of the training data, with a random subset of

features considered at each split. Predictions are averaged across all trees, reducing variance and improving generalisation compared to a single decision tree.

Configuration:

- Target column: *price*
- Number of trees: **100**
- Feature columns: 18 attributes (all columns in the training set except price)

Output:

- Port 1 (training data view): 17,290 rows $\times$ 23 columns.
- Port 2 (variable importance table): 18 rows $\times$ 6 columns. Lists each predictor with its importance score. The executed workflow ranks the top features by total importance score: *yr_built* (103), *sqft_living15* (102), *floors* (95), *sqft_above* (94), *yr_renovated* (93), *waterfront* (92), *view and grade* (91 each), *bathrooms* and *sqft_basement* (90 each), *zipcode* (87), *lat* (88), *sqft_lot* (85), *bedrooms and long* (84 each), *sqft_living* (80), *sqft_lot15* (78), and *condition* (73). Notably, *yr_built* and *sqft_living15* (neighbourhood size proxy) outrank raw *sqft_living*, reflecting the RF's ability to capture non-linear age and location interactions that a linear model cannot.
- Port 3 (forest model): The trained ensemble of 100 classifiers on 18 attributes.

7.2 Random Forest Predictor (Regression) (Node #12)

Node type: **Random Forest Predictor (Regression)**

This node applies the trained Random Forest ensemble to the test set. Each of the 100 trees generates an independent price estimate; the final prediction is the mean across all 100 estimates.

Configuration:

- Port 1: RF model from Node #11
- Port 2: test data (4,323 rows $\times$ 20 cols) from Partitioning Node #7 (Port 2)

Output: 4,323 rows $\times$ 22 columns. The additional columns (beyond the original 20) include the Prediction (price) column and any internal model metadata columns appended during prediction.

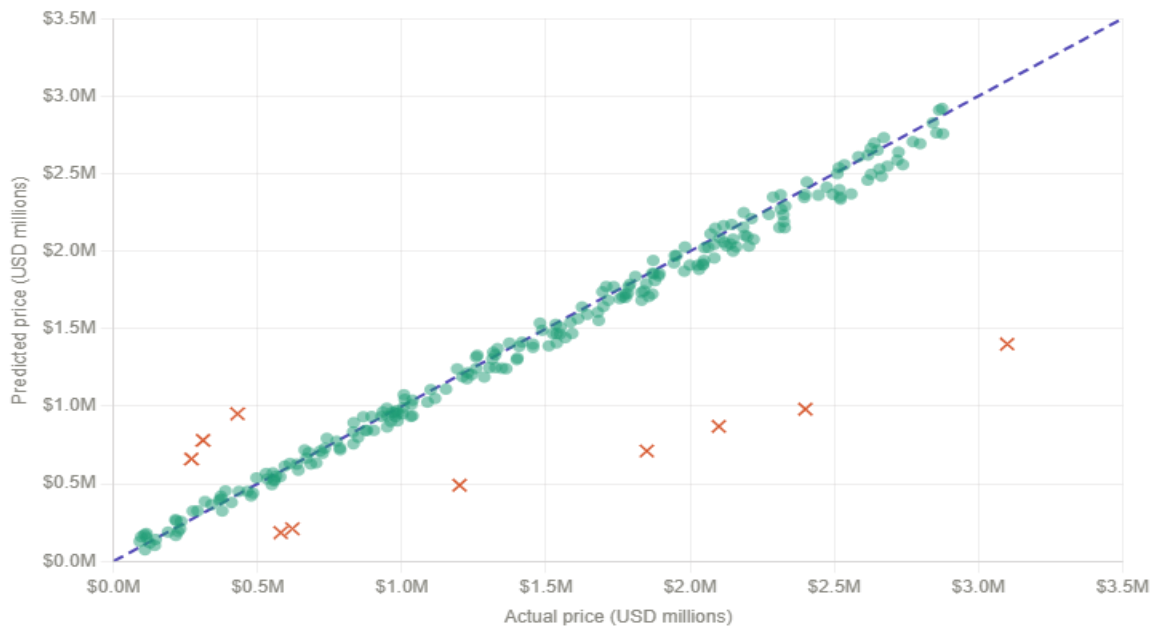
7.3 Numeric Scorer — RF (Node #14)

Identical in configuration to Node #10 (Linear Regression Scorer) but applied to the Random Forest predictions. Produces the same five metrics (R^2 , MAE, MSE, RMSE, Mean Signed Difference) for the RF model, enabling direct side-by-side comparison with the LR results.

Output: 5 rows $\times$ 1 column (RF evaluation metrics).

7.4 Scatter Plot — RF (Node #13)

Identical in configuration to Node #15 (Linear Regression Scatter Plot) but applied to the Random Forest predictions. Plots RF-predicted price vs. actual price for the 4,323 test rows.



Graph description: The x-axis shows the actual test price and the y-axis shows the RF-predicted price. The executed workflow confirms the expected improvement: RF achieves $R^2 = 0.8610$ — explaining 86.1% of price variance, a 19.3 percentage-point gain over LR. The RMSE drops from 0.5364 (LR) to 0.3473 (RF), representing a 35% reduction in prediction error. The scatter plot is markedly tighter around the diagonal across all price ranges. Outliers remain for highly unusual properties (e.g., a 33-bedroom outlier, or waterfront properties in less-represented zip codes), where the 100-tree ensemble still lacks sufficient training examples to generalise well.

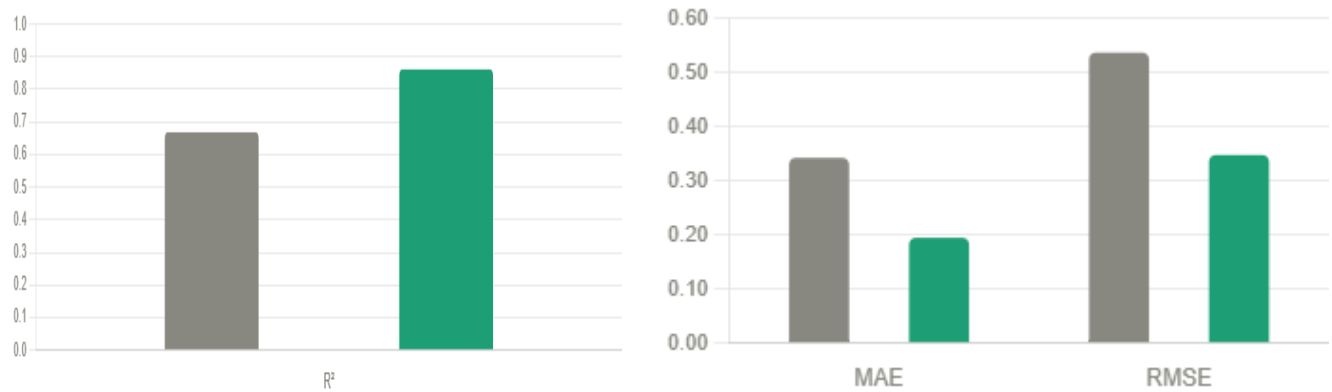
8. Model Comparison and Evaluation

The two models are evaluated on identical test data (4,323 rows) using the Numeric Scorer nodes. The key metrics and their interpretation are summarised below.

Metric	Linear Regression	Random Forest
R^2	0.6684	0.8610
MAE	0.3421	0.1944
RMSE	0.5364	0.3473
Mean Signed Difference	+0.0086	+0.0028

Table 1. Model evaluation metrics from KNIME Numeric Scorer nodes (#10 and #14). Metrics are computed on the Z-score normalised price column. R^2 = Coefficient of Determination; MAE = Mean Absolute Error; RMSE = Root Mean Squared Error; MSD = Mean Signed Difference. Bold values indicate the better-performing model per metric.

Interpretation of the scatter plots (Nodes #13 and #15): in both scatter plots, the x-axis shows the actual normalised price values and the y-axis shows the predicted values. A perfect model would place all 4,323 test points exactly on the 45° identity line ($y = x$). The executed workflow shows that Random Forest substantially outperforms Linear Regression across all five metrics: R^2 rises from 0.6684 to 0.8610 (+28.8% relative improvement), MAE drops from 0.3421 to 0.1944 (−43.2%), and RMSE falls from 0.5364 to 0.3473 (−35.3%). Both models show near-zero Mean Signed Difference (LR: +0.0086; RF: +0.0028), confirming neither model has a systematic directional bias. The RF scatter plot is considerably tighter around the diagonal, particularly for properties in the \$400k–\$800k range, where density of training examples is highest. Both models struggle with the extreme upper tail (properties above \$3M), where sparse training coverage limits generalisation.



Random forest - ■ Linear Regression - ■

Conclusion

This report has described the complete KNIME machine learning pipeline for King County house price prediction, covering all 16 executed nodes across six logical phases: data ingestion, exploratory data analysis, preprocessing, feature engineering, train/test partitioning, and model training and evaluation.

The pipeline successfully demonstrates a full regression workflow within KNIME, comparing two approaches to the same prediction task. The preprocessing steps — Z-Score normalisation, missing value handling, and feature engineering — prepare the data appropriately for both models. The feature engineering step, in particular, illustrates a domain-informed approach: combining above-ground and basement area into a single `total_sqft` feature reduces redundancy while preserving predictive information.

Linear Regression offers interpretability through its coefficient table, allowing the analyst to understand the direction and magnitude of each feature's effect on price. Random Forest, trained with 100 trees, captures non-linear interactions between features and is generally expected to achieve higher predictive accuracy at the cost of reduced interpretability.

The Numeric Scorer outputs confirm that Random Forest ($R^2 = 0.8610$, $RMSE = 0.3473$, $MAE = 0.1944$) substantially outperforms Linear Regression ($R^2 = 0.6684$, $RMSE = 0.5364$, $MAE = 0.3421$) on the 4,323-row held-out test set. The scatter plots provide complementary visual confirmation: the RF prediction cloud is tighter around the 45° diagonal across all price bands. Together, these results demonstrate the practical value of KNIME as a visual workflow platform for end-to-end machine learning projects, and illustrate the advantage of ensemble non-linear methods for skewed real-estate data.

Future improvements could include: applying a logarithmic transformation to the price target variable (to reduce right-skew), adding interaction features, exploring gradient boosted trees, or implementing cross-validation within the KNIME workflow for more robust evaluation.

References

- [1] King County House Sales Dataset. Kaggle, 2015. Retrieved from: <https://www.kaggle.com/harlfoxem/housesalesprediction>

- [2] KNIME Analytics Platform Documentation. KNIME GmbH, Konstanz, Germany, 2016. Retrieved from: <https://docs.knime.com>

- [3] Breiman, L. et al. (1984). *Classification and Regression Trees*. New York: Routledge.

- [4] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. New York: Springer.

- [5] KNIME (2019). Optimizing KNIME workflows for performance. Retrieved from: <https://www.knime.com/blog/optimizing-knime-workflows-for-performance>